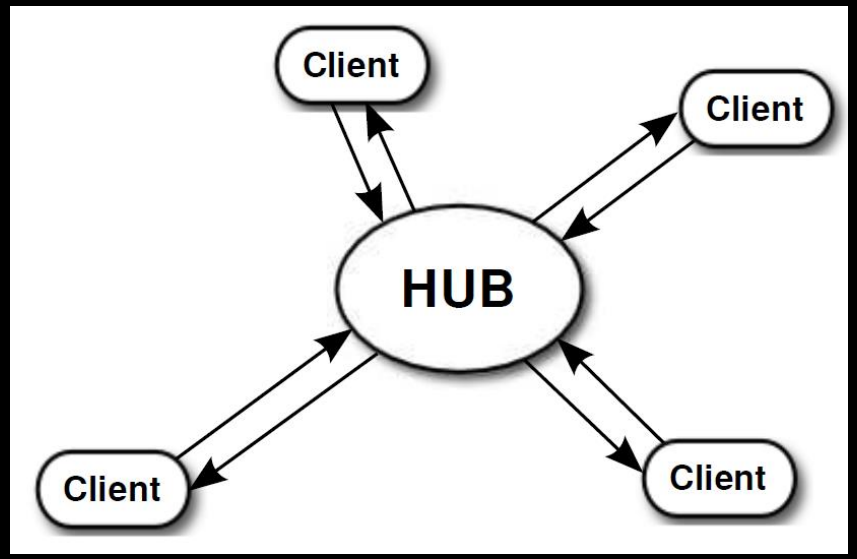
Sockets

- Two-way communication between networked computers
  - UDP – User Datagram Protocol
    - Connectionless and unreliable
  - TCP/IP – Transmission Control Protocol/Internet Protocol
    - Connection-oriented and reliable
  - Socket – generalized approach to connections
    - Server socket – listens for a connection
    - Client socket – makes the connection
  - Client/Server model
- Java.net package



Server Socket

- Open a port
  - Server creates a ServerSocket bound to a port
- Listen for connection requests
  - The server waits for clients using accept
  - The method blocks until a client tries to connect
- Communicate with the client
  - Use the input and output stream from the client socket
- Close connects when done

Client Socket

- Client socket
  - Attempts to connect to the server socket
  - Communicates with the server using the client sockets

Java Networking

- Ports
  - 0 – 1023 – well known for standard services
  - 1024 – 49151 – IANA assigned for specific applications or services
  - 49512– 65535 – dynamic/ephemeral ports
- Hosts
  - URL or URI
  - IPv4 addresses
  - IPv6 addresses
  - localhost
  - 127.0.0.1 (localhost)

- HTTP: 80 HTTPS: 443
- FTP: 21 FTPS/SSH: 22
- Telnet:23
- POP3: 110 POP3 SSL: 995
- IMAP: 143 IMAP SSL: 993
- SMTP: 25 (or 26) SMTP SSL: 587
- MySQL: 3306
- cPanel: 2082 cPanel SSL: 2083
- WHM: 2086 WHM SSL: 2087
- Webmail: 2095 Webmail SSL:2096
- WebDAV/WebDisk:2077
- WebDAV/WebDisk SSL: 2078

Java Networking – Server Side

```java
ServerSocket server = null;
try {
        server = new ServerSocket(4242);
        while (true) { // bad programming but we'll talk about this
                Socket conn = server.accept(); // block waits for a client
                service(conn); // perform the service for the client
                conn.close(); // close the socket when done
        }
} catch (IOException e) {
        e.printStackTrace();
}
```

Java Networking – Client Side

```
Socket conn = null;
try {
        conn = new Socket("192.168.0.1", 4242); // connect to the server
        request(conn); // request service
        conn.close(); // close the socket
} catch (IOException e) {
        e.printStackTrace();
}
```

How They Communicate

- Client connections almost always use ephemeral ports.
  - Example: A client connects from 127.0.0.1:58342 to server 127.0.0.1:5000.
  - 58342 is ephemeral, assigned dynamically.
- Server sockets usually listen on well-known or registered ports.
  - Example: HTTP server listens on 80 → clients connect from ephemeral ports.
- Tuple uniqueness:
  - The combination (clientIP, clientPort, serverIP, serverPort) is unique per connection.
  - Ephemeral ports allow multiple clients to connect to the same server port simultaneously.

How They Communicate

- Server is on: 192.168.1.10:5000
- Client is on: 192.168.1.20:58342
- The client connects to (192.168.1.10, 5000)
- The accept() creates a client socket specifically for the client
- The resulting tuple is (192.168.1.20, 58342, 192.168.1.10, 5000)
- The server uses the tuple to identify the connection
- All clients send the messages to (server ip, server port), and TCP/IP uses (client ip, client port) to identify which client socket gets the message on the server